

# Testing Guide

**Revision:** 5 **Last modified:** 2026-08-20T14:17:27Z

This is the **authoritative test-type catalogue** for qBittorrent-Fixed. Every testable module in the repo must have coverage in every applicable row below. The catalogue is derived from Part C of the completion-initiative plan (docs/superpowers/plans/2026-04-19-completion-initiative.md).

## Bootstrap (one-time virtualenv setup)

Test dependencies are **not** installed system-wide. Create and populate a local .venv/ (gitignored – see .gitignore) before running anything below:

```
python3 -m venv .venv
.venv/bin/pip install --upgrade pip
.venv/bin/pip install -r tests/requirements.txt
```

Verified working combination (2026-08-08): Python 3.14.6 (/usr/bin/python3.14) + pytest 9.1.1 + pytest-asyncio 1.4.0 + fastapi 0.141.1 + pydantic 2.13.4 + every package listed in tests/requirements.txt, all installed fresh into the same virtualenv (no cross-ABI mixing). Re-run the pip install line whenever tests/requirements.txt changes – it is idempotent.

Once bootstrapped, invoke pytest via .venv/bin/python -m pytest ... (or activate the venv first with source .venv/bin/activate and drop the .venv/bin/ prefix from every command in this guide).

## Test types

| # | Type                              | Framework                        | Directory                  | How to run                                           | Coverage / report land |
|---|-----------------------------------|----------------------------------|----------------------------|------------------------------------------------------|------------------------|
| 1 | Unit (Python)                     | pytest + pytest-asyncio          | tests/unit/                | pytest tests/unit/ -v --import-mode=importlib        | htmlcov/, coverage.xr  |
| 2 | Unit (Frontend)                   | Vitest + Angular Testing Library | frontend/src/**/*.spec.ts  | cd frontend && ng test                               | frontend/coverage/     |
| 3 | Integration (API)                 | pytest + httpx.AsyncClient       | tests/integration/         | pytest tests/integration/ -v --import-mode=importlib | htmlcov/, coverage.xr  |
| 4 | Integration (Plugin – i.e. nova3) | pytest + vcrpy (record/replay)   | tests/integration/plugins/ | pytest tests/integration/plugins/ -v                 | htmlcov/               |

| #  | Type                      | Framework                                     | Directory                                  | How to run                                                     | Coverage / report land      |
|----|---------------------------|-----------------------------------------------|--------------------------------------------|----------------------------------------------------------------|-----------------------------|
|    |                           |                                               | (pending Phase 4)                          |                                                                |                             |
| 5  | End-to-end (browser)      | Playwright (Python)                           | tests/e2e/                                 | pytest tests/e2e/ -v                                           | artifacts/e2e/              |
| 6  | End-to-end (API flow)     | pytest + live compose                         | tests/e2e/ (fixture-gated)                 | pytest tests/e2e/ -v --import-mode=importlib                   | htmlcov/                    |
| 7  | Contract (OpenAPI)        | schemathesis                                  | tests/contract/                            | schemathesis run http://localhost:7187/openapi.json            | artifacts/contract/         |
| 8  | Property-based hypothesis |                                               | tests/property/ (pending)                  | pytest tests/property/ -v                                      | htmlcov/                    |
| 9  | Fuzz                      | hypothesis + atheris (optional)               | tests/fuzz/ (pending)                      | pytest tests/fuzz/ -v                                          | artifacts/fuzz/             |
| 10 | Mutation                  | mutmut                                        | runs against download-proxy/src/ (pending) | mutmut run                                                     | html/mutmut.html            |
| 11 | Security                  | pytest + bandit + semgrep rules               | tests/security/                            | pytest tests/security/ -v                                      | htmlcov/ + artifacts/scans/ |
| 12 | Performance (latency)     | pytest-benchmark                              | tests/performance/, tests/benchmark/       | pytest tests/performance/ tests/benchmark/ -v --benchmark-only | .benchmarks/                |
| 13 | Load (throughput)         | Locust                                        | tests/load/ (pending Phase 6)              | locust -f tests/load/locustfile.py                             | artifacts/load/             |
| 14 | Stress (overload)         | Locust + chaos probes                         | tests/stress/                              | pytest tests/stress/ -v                                        | artifacts/stress/           |
| 15 | Chaos (fault injection)   | toxiproxy + pytest                            | tests/chaos/ (pending)                     | pytest tests/chaos/ -v                                         | artifacts/chaos/            |
| 16 | Concurrency (race)        | pytest-randomly + pytest-repeat + asyncio ops | tests/concurrency/ (pending)               | pytest tests/concurrency/ -v --count=10                        | htmlcov/                    |
| 17 | Memory leak               | tracemalloc + pytest + objgraph               | tests/memory/ (pending)                    | pytest tests/memory/ -v                                        | artifacts/memory/           |
| 18 | Deadlock / timeout        | pytest-timeout (hard cap all tests)           | global                                     | pytest -v (30 s timeout is the default in pyproject.toml)      | test log                    |
| 19 | Smoke                     | pytest                                        |                                            |                                                                | htmlcov/                    |

| #  | Type                     | Framework                                        | Directory                      | How to run                                                    | Coverage / report land                    |
|----|--------------------------|--------------------------------------------------|--------------------------------|---------------------------------------------------------------|-------------------------------------------|
|    |                          |                                                  | tests/smoke/ (pending)         | pytest tests/smoke/ -v                                        |                                           |
| 20 | Monitoring / metrics     | pytest + Prometheus scrape asserts               | tests/observability/ (pending) | pytest tests/observability/ -v                                | htmlcov/                                  |
| 21 | Infra / compose          | pytest-docker + compose-config validator         | tests/infra/ (pending)         | pytest tests/infra/ -v                                        | htmlcov/                                  |
| 22 | Accessibility            | Playwright + axe-core                            | tests/ally/ (pending)          | pytest tests/ally/ -v                                         | artifacts/ally/                           |
| 23 | Visual regression        | Playwright snapshots                             | tests/visual/ (pending)        | pytest tests/visual/ --update-snapshots                       | tests/visual/ __snapshots/                |
| 24 | Documentation link check | pytest + linkchecker                             | tests/docs/ (pending)          | pytest tests/docs/ -v                                         | artifacts/docs/                           |
| 25 | Type-check (static)      | mypy --strict + tsc --noEmit                     | CI job                         | mypy download-proxy/src plugins / cd frontend && tsc --noEmit | CI log                                    |
| 26 | Lint (static)            | ruff + eslint + shellcheck + hadolint + yamllint | CI job                         | ruff check ./ cd frontend && ng lint / shellcheck *.sh        | CI log                                    |
| 27 | Dependency audit         | pip-audit + npm audit + Snyk + Trivy             | CI job                         | pip-audit -r download-proxy/requirements.txt / snyk test      | artifacts/scans/                          |
| 28 | SAST                     | Semgrep + Bandit + SonarQube                     | CI job                         | semgrep scan --config=auto / bandit -r download-proxy/src     | artifacts/scans/                          |
| 29 | Secret scan              | Gitleaks + TruffleHog                            | CI job                         | gitleaks detect                                               | artifacts/scans/                          |
| 30 | License compliance       | pip-licenses + license-checker                   | CI job                         | pip-licenses / cd frontend && license-checker                 | artifacts/licenses/                       |
| 31 | DDoS (abuse resilience)  | pytest + Starlette TestClient + httpx ASGI       | tests/ddos/                    | pytest tests/ddos/ -v --import-mode=importlib                 | htmlcov/ -- see <a href="#">DDoS test</a> |

Rows marked *pending* are scaffolded by the completion-initiative plan phases (see the plan document for the phase that ships each one).

## Where tests live

- **Top-level tests** / " all merge-service tests, *not* download-proxy/ tests/ (which does not exist).
- **frontend/src/\*\*/\*.spec.ts** " co-located Angular unit tests.
- **CI jobs** " .github/workflows/{syntax,unit,integration,nightly,security}.yml.

## Service-availability fixtures

Since Phase 0 the following fixtures **start the compose stack** rather than skip when services are down (converted 71 runtime skips to fixture-gated executions, per commit 55d29ce):

| Fixture            | Starts            | Asserts                                   |
|--------------------|-------------------|-------------------------------------------|
| merge_service_live | qbittorrent-proxy | GET http://localhost:7187/health<br>" 200 |
| qbittorrent_live   | qbittorrent       | GET http://localhost:7186/ " 200          |
| webui_bridge_live  | webui-bridge.py   | GET http://localhost:7188/health<br>" 200 |
| all_services_live  | all three         | all of the above                          |

Defined in tests/fixtures/services.py and wired into tests/conftest.py.

## How to add a new test (TDD cadence)

The CLAUDE.md critical constraint is non-negotiable:

1. **RED** " write the failing test first. Name it after the behaviour, not the implementation. Put it in the directory that matches the test type from the catalogue above.
2. **Watch it fail** " run the test in isolation (pytest tests/unit/test\_new\_thing.py::test\_specific\_case -v) and confirm the failure reason is the one you want (NOT an import error, NOT a fixture error).
3. **GREEN** " write the minimum production code to make the test pass. No refactors yet.
4. **Verify** " run the whole affected test directory to prove nothing else regressed.
5. **Rebuild-reboot** " if the change touched container code, follow the CLAUDE.md REBUILD AND REBOOT constraint:

```
./stop.sh
podman exec qbittorrent-proxy find / -name __pycache__ -type d -exec rm -rf {} \;
./start.sh -p
```

Curl-check the served content matches the committed code.

6. **Commit** “one behaviour per commit. Commit message follows `<type>(phase-N): <subject>` when part of a phased plan.

## Coverage gate

- `pyproject.toml` contains `[tool.coverage.report]` with `fail_under` set to the current gate (Phase 0 starts low; Phase 5 raises it module-by-module to 100 %).
- The baseline zero point is captured in [COVERAGE\\_BASELINE.md](#).
- HTML reports land in `htmlcov/`; XML (for SonarQube) in `coverage.xml`.

## CI entry points

- **Local full pipeline** “`./ci.sh`
- **Quick loop** “`./ci.sh --quick`
- **Hardcoded-podman runner** “`./run-all-tests.sh` (note: hardcodes podman, will fail on docker-only systems “see [CLAUDE.md](#) gotcha)
- **Single-file** “`./test.sh --plugin <name>` or `./test.sh --full`

## Per-tracker search stats

Tests that cover `TrackerSearchStat` and the `tracker_started / tracker_completed` SSE events live across four suites:

| Test file                                                   | Type             | What it pins                                                                                                                                                                                                                                                                                                        |
|-------------------------------------------------------------|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>tests/unit/merge_service/test_tracker_stats.py</code> | Unit             | <code>TrackerSearchStat</code> defaults, status transitions (pending “ running “ success / empty / error / timeout), <code>duration_ms</code> bookkeeping, <code>auth_flag</code> provenance, and sorted <code>to_dict()</code> serialisation. The SSE poll loop emits <code>tracker_started</code> exactly once on |
| <code>tests/unit/api_layer/test_tracker_stats_sse.py</code> | Unit (streaming) | pending “ running and <code>tracker_completed</code> exactly once per terminal flip; gracefully degrades                                                                                                                                                                                                            |

| Test file                                           | Type                     | What it pins                                                                                                                                                                                                                                                                                                                                                                                                           |
|-----------------------------------------------------|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| tests/property/<br>test_tracker_stats_properties.py | Property<br>(Hypothesis) | when metadata predates<br>tracker_stats.<br>sum of results_count ==<br>total_results, every<br>completed stat has<br>duration_ms >= 0,<br>set(tracker_stats<br>keys) ==<br>set(trackers_searched),<br>failed trackers retain<br>error_type + error.<br>POST /api/v1/search,<br>POST /api/v1/search/<br>sync, and GET /api/v1/<br>search/{id} all expose<br>the 15-field<br>tracker_stats payload<br>on SearchResponse. |
| tests/contract/<br>test_tracker_stats_contract.py   | Contract                 |                                                                                                                                                                                                                                                                                                                                                                                                                        |

The frontend dialog + chip bar are covered by frontend/src/app/components/tracker-stat-dialog/tracker-stat-dialog.component.spec.ts and the tracker\_stats bar describe block in frontend/src/app/components/dashboard/dashboard.component.spec.ts.

## Runtime theme (palette system)

The runtime-switchable palette system (see [docs/DESIGN\\_SYSTEM.md](#)) is guarded by a four-tier test bundle “every tier asserts *observable* state (DOM, CSS variables, localStorage, Playwright computed style) per the “no false positives” mandate:

| Test file                              | Type               | What it pins                                                                                                                                                                                                                                                                                                                                                                                                     |
|----------------------------------------|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| tests/unit/<br>test_palette_catalog.py | Unit (parametric)  | Every palette has both<br>variants, every one of the<br>15 tokens is a valid CSS<br>colour, ids are unique,<br>DEFAULT_PALETTE_ID<br>resolves, Darcula accent<br>is #9d001e. Per-palette<br>parametrisation names<br>the offender.<br>:root fallback block<br>covers every token;<br>dashboard SCSS uses<br>var(--color-*) instead<br>of \$accent/\$bg-* Sass<br>variables; dashboard<br>template includes <app- |
| tests/unit/<br>test_theme_wiring.py    | Unit (integration) |                                                                                                                                                                                                                                                                                                                                                                                                                  |

| Test file                                                               | Type                    | What it pins                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-------------------------------------------------------------------------|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| frontend/src/app/services/theme.service.spec.ts                         | Unit (Vitest)           | <p>theme-picker&gt;;<br/>           TOKEN_CSS_VAR map covers every var.</p> <p>System-dark detection, stored-state hydration, unknown-id fallback, setPalette writes all 15 tokens with exact values to<br/>           documentElement.style, toggleMode flips signal + data-mode + color-scheme, prefers-color-scheme change listener respects<br/>           modeIsUserChosen flag.</p> <p>One menu item per palette, swatches render with accent/contrast/bg/text colours, clicking an item calls setPalette, clicking outside closes the menu, active palette has .active + aria-checked="true".</p> <p>Live dashboard at :7187: picking a new palette changes --color-accent (+3 others) on &lt;html&gt; to the catalogue value AND writes qbit.theme into localStorage. Skips with a loud message when the served bundle predates the theme-picker.</p> |
| frontend/src/app/components/theme-picker/theme-picker.component.spec.ts | Unit (Vitest)           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| tests/e2e/test_theme_runtime.py                                         | End-to-end (Playwright) |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |

Whenever the TrackerSearchStat shape changes, run ./scripts/freeze-openapi.sh to refresh docs/api/openapi.json so the contract test test\_frozen\_and\_live\_have\_same\_schemas stays green.

## DDoS tests

tests/ddos/ is the  $\S 11.4.27(B)$  /  $\S 11.4.169$  **ddos tests** type: abuse resilience, as distinct from throughput (tests/load/), overload (tests/stress/) and threshold correctness (tests/security/).

## What it covers

| Dimension                             | File                        | The question it answers                                                                                                                                                                                                                                                               |
|---------------------------------------|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Capped burst flood                    | test_request_flood.py       | Are excess requests <b>refused</b> rather than served? Does the refusal path stay clean (no 5xx)? Does the service still answer afterwards? Can one flooding client deny service to <b>another</b> client? Does one slow request <b>starve</b> everyone else (head-of-line blocking)? |
| Slow client / slow body               | test_slow_request.py        | Covers both a slow handler and a dribbled request body.                                                                                                                                                                                                                               |
| Payload-size abuse                    | test_payload_abuse.py       | Is an oversized upload <b>refused by an explicit guard</b> (413) rather than buffered and processed? Does an oversized JSON body avoid a 5xx? Can size be used to <b>bypass</b> the per-IP limiter?                                                                                   |
| Resource-exhaustion admission control | test_resource_exhaustion.py | Does MAX_CONCURRENT_SEARCHES actually <b>engage</b> and shed load with a 429? Does it catch a distributed client whose per-IP budget is untouched?                                                                                                                                    |

Every assertion reads a **user-observable outcome** “a status-code population, a response body, a post-attack liveness probe, bystander throughput” never “no exception was raised” (Â§11.4 / Â§11.4.1).

**Evidence class (Â§11.4.226):** runtime-on-the-real-ASGI-app. The tests drive the real `api.app` through the real middleware stack, real routers and real `slowapi` decorators, **in-process**.

## Safety bounds (Â§12, Â§12.6, Â§12.11, CLAUDE.md Mandatory Standard 9)

This host runs live containers and parallel agents, and resource exhaustion here has previously caused forced logouts. The suite is therefore bounded by construction, not by convention. **No bound below may be raised without re-justifying the host budget here.**

| Bound                | Value                                   | Why                                                                                                                                                    |
|----------------------|-----------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| Live stack contact   | <b>none</b>                             | No socket is opened. Nothing ever reaches 127.0.0.1:7187 or any port. A flood against the operator's running stack is forbidden outright.              |
| Largest single burst | MAX_BURST_REQUESTS = <b>24</b>          | ~4x the smallest limit under test "enough to prove refused, not served" with margin. There is no unbounded loop and no flood until it breaks anywhere. |
| Concurrent clients   | MAX_CONCURRENT_CLIENTS = <b>5</b>       | Coroutines on ONE event loop. No thread pool, no process pool.                                                                                         |
| Liveness probes      | MAX_LIVENESS_PROBES = <b>60</b> @ 20 ms | Sub-millisecond in-process GET / health calls with no I/O.                                                                                             |
| Largest allocation   | OVERSIZE_UPLOAD_BYTES ^ <b>10 MiB</b>   | Crosses the production 10 MiB guard by 4 KiB "the smallest margin that proves it."                                                                     |
| Tracker fan-out      | <b>neutralised</b>                      | SearchOrchestrator._run_search is stubbed, so no test can emit traffic at RuTracker / Kinozal / NNMClub or spawn nova3 subprocesses.                   |
| Wall clock           | whole suite ^ <b>27 s</b>               | Well inside the 60 s pytest-timeout cap.                                                                                                               |

The constants live in tests/ddos/conftest.py and are read by the tests "they are hard caps, not targets."

## What this does NOT cover (honest gaps, §11.4.6 / §11.4.3)

Stated so nobody mistakes a green run for more than it is:

1. **TCP-level slowloris is not tested anywhere.** Partial headers, a dribbled TLS handshake, kernel accept-queue exhaustion and uvicorn's own connection / keep-alive timeouts all live *below* ASGI and are unreachable in-process. test\_slow\_request.py covers the ASGI-layer analogue (a dribbled body); the socket-level probe is covered by **neither** this suite nor challenges/scripts/ddos\_resilience\_challenge.sh. Genuinely open.
2. **No multi-host or real-network traffic.** Distributed source addresses are *simulated* via X-Forwarded-For with the production TRUST\_FORWARDED\_FOR knob enabled. Real botnet behaviour, NAT effects and upstream infrastructure are out of scope.
3. **Saturation is simulated, not driven.** test\_resource\_exhaustion.py sets the orchestrator's in-flight counter directly rather than starting real 43-tracker fan-outs "driving true saturation is exactly the off-

host damage the host budget forbids. The *guard* under test is real; the *state* is set.

4. **SearchRequest.query has no max\_length** (api/routes.py:187 declares only min\_length=1). An oversized query is therefore **accepted** and forwarded to the tracker fan-out “a real amplification gap. test\_oversized\_query\_does\_not\_5xx\_and\_service\_stays\_live pins the properties that *do* hold (no 5xx, service stays live) rather than asserting a refusal that does not exist. Closing the gap is a production-source change and is **not** done by this suite.
5. **Multi-worker / multi-process limiter coherence is untested.** The default memory:// slowapi backend is per-process; a deployment scaled past one worker needs RATE\_LIMIT\_STORAGE\_URI pointed at Redis for the counters to mean anything. Nothing here exercises that path.

## How to run

```
# the suite (in-process, ~27s, safe to run any time)
.venv/bin/python -m pytest tests/ddos/ -v --import-mode=importlib
```

```
# the paired “1.1 falsifiability proof” breaks each defence in a THROW
# COPY of download-proxy/src and asserts the suite turns red
bash tests/ddos/mutations/run_mutation_check.sh
```

The mutation runner never touches the real source tree: it copies download-proxy/src to a mktemp -d directory, mutates the copy, and points the suite at it via the BOBA\_DDOS\_SRC\_PATH seam in tests/ddos/conftest.py. This checkout is worked by several agents at once, so an in-place mutation could be swept into someone else’s commit before restoration the 11.4.84 working-tree-quiescence failure mode. It also runs an **unmutated control** first, so a failure caused by the copy mechanism can never be mistaken for a mutation being caught (11.4.201(1)).

Mutations and the defence each one breaks:

| #  | Mutation                                                           | Must turn red               |
|----|--------------------------------------------------------------------|-----------------------------|
| M1 | Rate limits raised to 100000/minute                                | test_request_flood.py       |
| M2 | Limiter key collapsed to a constant (all clients share one bucket) | test_request_flood.py       |
| M3 | Upload size guard raised 10 MiB → 10 GiB                           | test_payload_abuse.py       |
| M4 | is_search_queue_full() → always False                              | test_resource_exhaustion.py |
| M5 | Synchronous time.sleep in the request path (head-of-line blocking) | test_slow_request.py        |

**M5 is why the slow-request oracle is throughput and not latency.** Two earlier versions of that file were bluffs that M5 caught: asserting *ordering* stayed green because a blocking request still finishes last, and asserting per-

probe *latency* stayed green because a blocked event loop does not make a bystander probe slow – it stops scheduling it at all (measured: worst probe latency 0.0129 s while only **2** probes ran in a 0.948 s window). Bystander **throughput** is the observable that actually collapses.

## Relationship to the neighbouring suites

tests/ddos/ deliberately does not re-test what these already cover:

- tests/unit/test\_rate\_limit.py – slowapi works, per-IP isolation, 429 body, on a **synthetic** FastAPI app.
- tests/security/test\_rate\_limit\_public\_endpoints.py – the **real** app’s routes carry the limiter at their **configured thresholds** (§11.4.196(F)).
- challenges/scripts/ddos\_resilience\_challenge.sh – the **live-socket** layer: a bounded localhost concurrency ramp against :7185 / :7187 / :7189 health surfaces, crash resistance, cross-endpoint isolation, and the BOB-112 /healthz TTL-cache regression guard. Requires the stack to be up; this pytest suite does not.

There is one deliberate overlap:

test\_flooding\_client\_does\_not\_deny\_service\_to\_another\_client asserts per-client isolation that the unit suite also asserts – but on the **real** app, so it notices a production route or middleware regression that collapses the key function, which a synthetic-app test structurally cannot.

## Gotchas

- pytest needs --import-mode=importlib for the merge service – the project is not a Python package (no \_\_init\_\_.py at the root).
- Integration tests with requests.get("http://localhost:7187/").ok guards are **obsolete** since Phase 0 – use the live fixtures instead. tests/unit/test\_no\_runtime\_service\_skips.py enforces this.
- pytest-timeout hard-caps every test at 30 seconds. Stress and load tests that legitimately run longer must override with @pytest.mark.timeout(...).
- The CI workflow is **no longer manual-only** since Phase 0.4 – push and PR triggers the syntax / unit / integration workflows. Only the nightly.yml and security.yml are scheduled.

## Test counts (machine-derived, §11.4.259)

Regenerated by [scripts/compute-badges.sh](#) – never hand-typed.  
Corroborates the README badge row + the Contributing-section bullets.  
Last regenerated: 2026-08-20T15:15:27Z.

| Suite                      | Directory | Real count  | Method                   |
|----------------------------|-----------|-------------|--------------------------|
| Python (whole tests/ tree) | tests/    | <b>5400</b> | pytest --collect-only -q |

| Suite                | Directory                         | Real count | Method                                  |
|----------------------|-----------------------------------|------------|-----------------------------------------|
| Python unit          | tests/unit/                       | 4406       | pytest --collect-only -q                |
| Python integration   | tests/integration/                | 288        | pytest --collect-only -q                |
| Python e2e           | tests/e2e/                        | 30         | pytest --collect-only -q                |
| Python contract      | tests/contract/                   | 12         | pytest --collect-only -q                |
| Frontend (Vitest)    | frontend/src/**/*.spec.ts         | 371        | collected (vitest list --run)           |
| HelixQA Challenges   | challenges/scripts/*.sh           | 38         | ls challenges/scripts/*.sh<br>  wc -l   |
| Pre-build invariants | scripts/pre_build_verification.sh | 44         | max total of every [N/N] progress label |

**BOB-118 provenance note:** the README badge row previously read python tests-585 passing / frontend tests-182 passing with no corroborating source anywhere in this document (a “11.4.6 bluff” the numbers were off by roughly 9x and could not be traced to any real invocation). This table is that corroborating source going forward; wording changed from “passing” to “collected” because collection counts (this table’s method) prove existence, not that every test currently passes a materially different, honestly-labelled claim.